

第4章 JavaScript 的用法

参考网址:

<https://www.w3school.com.cn/>

4.1 概述

4.1.1 第一个 JavaScript 程序

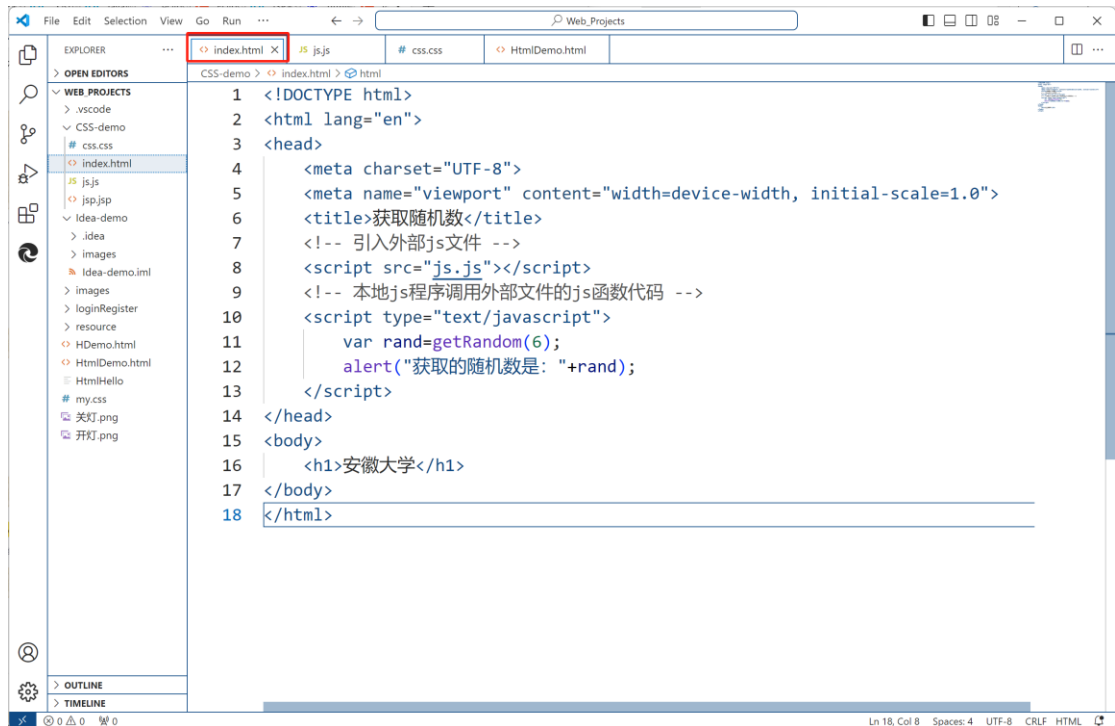
在页面中弹出一个 $1+1=3$? 是否正确的例子。代码如下:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>安徽大学</title>
  <script type="text/javascript">
    var a=1;
    var b=1;
    var sum=a+b;
    var c=confirm("1+1="+sum+",是否正确? ")
    alert("非常正确! ");
  </script>
</head>
<body>
  <h1>安徽大学</h1>
</body>
</html>
```

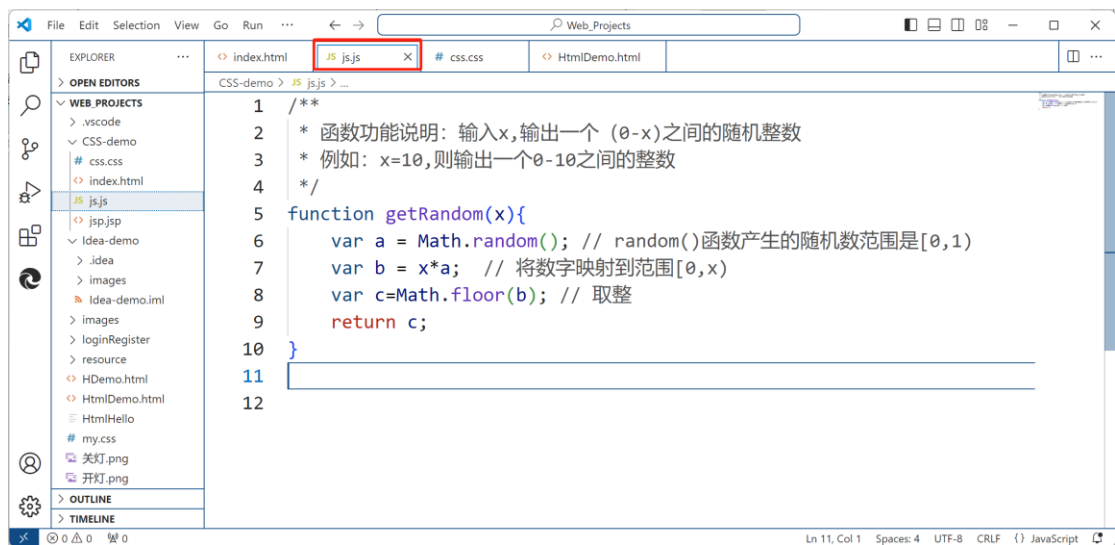
4.1.2 引入外部的 js 程序

语法格式: `<script type = "text/javascript" src = "js 文件路径"/></script>`

1、首先编写 html 文件,代码如下:



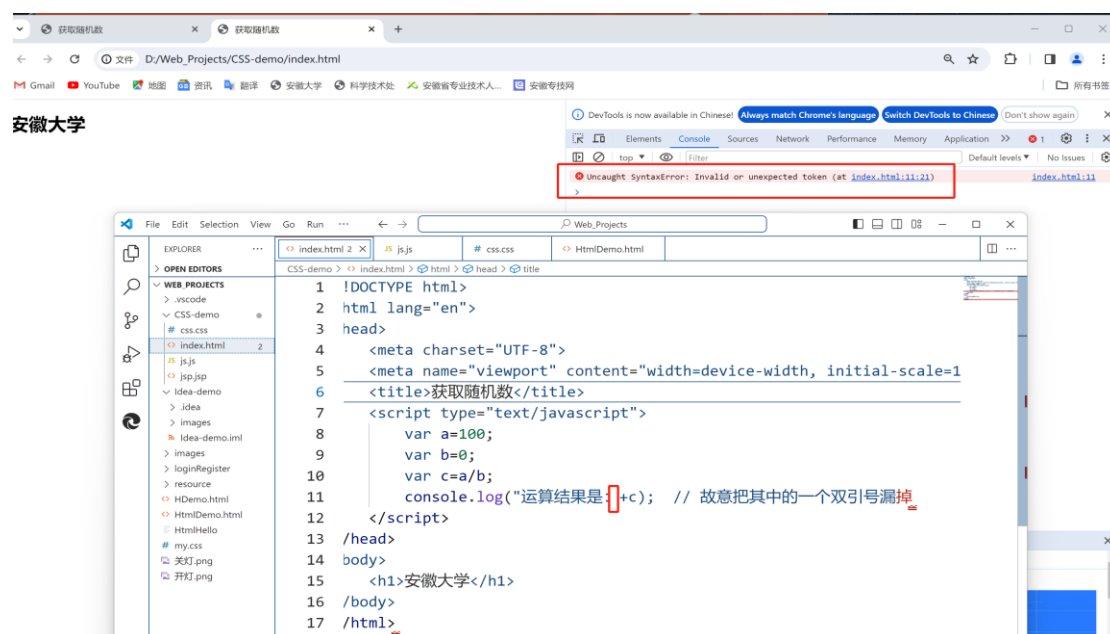
1、接着创建新的.js 文件，然后输入以下代码：



4.1.3 错误调试

编写一段程序代码，故意把 js 代码中一个括号漏掉，运行程序，调出浏览器。

接着按住 f12(部分电脑是 Fn+F12)调出浏览器的开发者模式。



4.2 语法规则

见 PPT。

4.3 函数

函数的实质就是可以作为一个逻辑单元对待的一组 JavaScript 代码。使用函数可以使代码更为简洁，提高重用性。在 Javascript 中，大约 95%的代码都包含在函数中，由此可见，函数的重要性。

4.3.1 函数的定义

函数是由关键字 `function`、函数名加一组参数以及置于大括号中需要执行的一段代码定义。定义的基本语法如下：

```
function name( [para1, para2, para3,...] ){  
    statements;  
    [ return expression ]  
}
```

参数说明：

- **name:** 函数名，必选项，在同一个页面中函数名必须唯一，并且区分大小写。
- **para:** 函数参数，是可选项，用于指定参数列表，当使用多个参数时，参数间使用逗号隔开。一个函数最多可以有 255 个参数。

- **statements:** 必选项，是函数体，用于实现函数功能的语句。
- **返回值:** 可选项，是函数体用于实现函数功能的语句。

例题: 用于获取一个 0-x 的随机数函数。

```
/**
 * 函数功能说明: 输入 x, 输出一个 (0-x) 之间的随机整数
 * 例如: x=10, 则输出一个 0-10 之间的整数
 */
function getRandom(x){
    var a = Math.random(); // random() 函数产生的随机数范围是[0,1)
    var b = x*a; // 将数字映射到范围[0,x)
    var c=Math.floor(b); // 取整
    return c;
}
```

4.3.2 函数的调用

函数的调用比较简单，如果调用不带参数的函数，使用函数名加上括号就行；如果调用的函数带参数，则在括号中加上需要传递的参数；如果函数有返回值，则可以使用赋值语句将函数传给一个变量。

例题: 定义一个 Javascript 函数 checkName(), 用于验证输入的名字是否合法。



代码如下:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>检查输入的姓名是否合法</title>
    <script type="text/javascript">
        function checkName() {
            var str = form1.realName.value; //获取输入的真实姓名
            if (str == "") { //当真实姓名为空时
                alert("请输入真实姓名!");
                form1.realName.focus(); // 将光标放置在输入框
            }
            return;
        }
    </script>
</head>
<body>
    请输入真实姓名: <input type="text" value="张三" /> <button value="检测" />
</body>
</html>
```

```

    } else {
        //当真实姓名不为空时
        var objExp = /[u4E00-u9FA5]{2,}/; //创建 RegExp 对象
        if (objExp.test(str) == true) { //判断是否匹配
            alert("您输入的真实姓名正确！");
        } else {
            alert("您输入的真实姓名不正确！");
        }
    }
}
</script>
</head>
<body>
    <center>
        <form name="form1" method="post" action="">
            请输入真实姓名: <input name="realName" type="text"
id="realName">
            <input name="Button" type="button" onClick="checkName()"
value="检测">
        </form>
    </center>
</body>
</html>

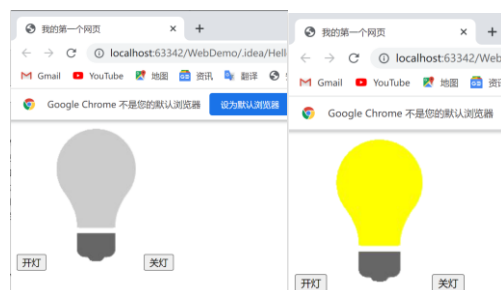
```

4.5 事件处理

Javascript 与 Web 页面之间的交互是通过用户操作浏览器页面时触发相关事件来实现的，例如在页面加载完毕时触发 `onload()` 事件，当用户单击按钮时将触发按钮的 `onclick` 事件。事件处理程序用于响应某个事件而执行的处理程序。事件处理程序可以是任意 JavaScript 语句，但通常使用特定的自定义函数 `Function` 来对待事件进行处理。

4.5.1 事件处理举例

例题：通过设置 2 个按钮，实现开灯和关灯操作。



```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">

```

```

    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>开关灯测试</title>
</head>
<body>
    <input type="button" onclick="on()" value="开灯">
    
    <input type="button" onclick="off()" value="关灯">
    <script>
        function on() {
            document.getElementById("myImage").src = "../images/开
灯.png";
        }
        function off() {
            document.getElementById("myImage").src = "../images/关
灯.png";
        }
    </script>
</body>
</html>

```

4.5.2 键盘事件

键盘控制图片移动的应用实例。

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>键盘移动图片</title>
    <style>
        #container {
            position: relative; width: 500px;
            height: 500px; border: 1px solid #000;
        }
        #moveableImage {
            position: absolute; width: 100px; height: 100px;
        }
    </style>
</head>
<body>
    <div id="container" name="d1">

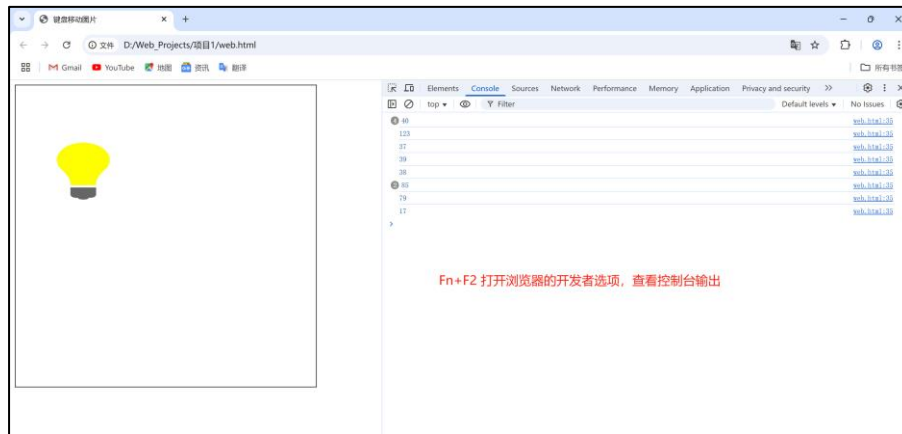
```

```

        
    </div>
    <script>
        document.addEventListener('keydown', function (event) {
            const img = document.getElementById('moveableImage');
            const step = 10; // 移动的像素值
            // 获取图片当前位置和尺寸
            let rect = img.getBoundingClientRect();
            let newX = rect.left;
            let newY = rect.top;
            // 打印按键的 ascii 码
            var key=event.keyCode;
            console.log(key);
            switch (event.key) {
                case 'ArrowLeft': // 左箭头键
                    newX -= step;
                    break;
                case 'ArrowRight': // 右箭头键
                    newX += step;
                    break;
                case 'ArrowUp': // 上箭头键
                    newY -= step;
                    break;
                case 'ArrowDown': // 下箭头键
                    newY += step;
                    break;
                default:
                    return; // 如果不是方向键，不执行任何操作
            }
            img.style.left = newX + 'px'; // 更新图片的 left 属性以移动图片
            img.style.top = newY + 'px'; // 更新图片的 top 属性以移动图片
        });
    </script>
</body>
</html>

```

运行效果如下：

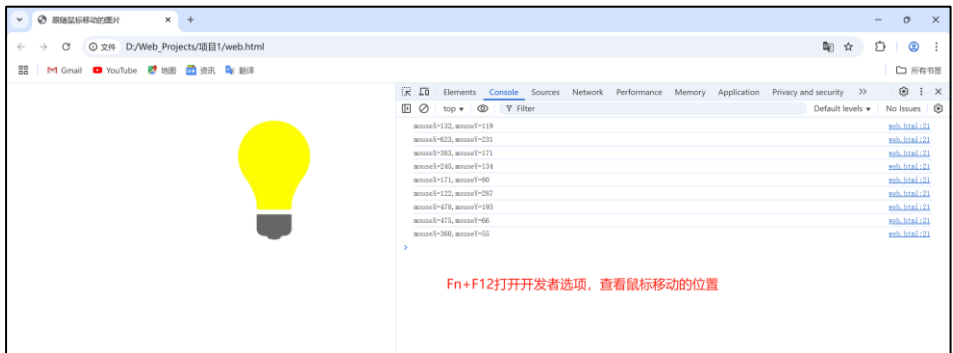


4.5.3 鼠标事件

跟随鼠标移动的图片应用实例。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>跟随光标移动的图片</title>
  <style>
    #image {
      position: absolute;
      /* 确保图片不影响鼠标事件 */
      pointer-events: none;
    }
  </style>
</head>
<body>
  
  <script>
    window.onload = function () {
      var im = document.getElementById("image");
      function moveImage(e) {
        im.style.left = e.pageX + 'px';
        im.style.top = e.pageY + 'px';
        console.log("mouseX="+e.pageX+",mouseY="+e.pageY);
      }
      document.addEventListener('mousemove', moveImage);
    }
  </script>
</body>
</html>
```


运行效果如下：



函数说明：document.addEventListener(event, function, useCapture)

参数	描述
<i>event</i>	必需。 描述事件名称的字符串。 不要使用 "on" 前缀。例如，使用 "click" 来取代 "onclick"。
<i>function</i>	必需。 描述事件触发后执行的函数。 当事件触发时，事件对象会作为第一个参数传入函数。
<i>useCapture</i>	可选。 布尔值，指定事件是否在捕获或冒泡阶段执行。 可能值： • true - 事件句柄在捕获阶段执行 • false- 默认。事件句柄在冒泡阶段执行

使用 removeEventListener()方法移除通过 addEventListener()方法添加的事件句柄。

4.5.4 常用事件表

JavaScript 常用事件下：

事件类型	事件	说明
表单相关事件	onfocus	当某个元素获得焦点时触发此事件。
	onblur	当某个元素失去焦点时触发此事件。
	onchange	当某个元素失去焦点并且元素的内容发生改变时触发此事件。
	onsubmit	提交表单时触发此事件。
	onreset	表单被重置时触发此事件。
鼠标键盘 相关事件	onclick	单击鼠标时触发此事件。
	ondblclick	双击鼠标时触发此事件。
	onmousedown	按下鼠标时触发此事件。
	onmouseup	按下鼠标后松开鼠标时触发此事件。
	onmouseover	当鼠标移动到某元素的区域时触发此事件。
	onmousemove	当鼠标在某元素的区域内移动时触发此事件。
	onmouseout	当鼠标离开某元素的区域时触发此事件。
	onkeypress	键盘上的键被按下并释放时触发此事件。可处理单键的操作。
	onkeydown	键盘上的键被按下时触发此事件。可处理单键或组合键的操作。
页面相关事件	onkeyup	键盘上的键被按下后松开时触发此事件。可处理单键或组合键的操作。
	onload	当页面完成加载时触发此事件。
	onunload	当离开页面时触发此事件。
	onresize	当窗口大小改变时触发此事件。

尝试使用鼠标和键盘事件，控制页面中的元素移动。例如：俄罗斯方块类的游戏开发，扑

克牌游戏开发，跟随鼠标移动的小球等。

4.6 常用对象

4.6.1 window 对象

window 对象即浏览器窗口对象，是一个全局对象，是所有对象的顶级对象，在 JavaScript 中起着举足轻重的作用。

(1) window 对象的属性

属性	说明
document	对话框中显示的当前的文档
frames	表示当前对话框中所有frame对象的集合
location	指定当前文档的URL
name	对话框的名字
status	状态栏中的当前信息
defaultstatus	状态栏的默认信息
top	表示最顶层的浏览器对话框
parent	表示包含当前对话框的父对话框
opener	表示打开当前对话框的父对话框
closed	表示当前对话框是否关闭的逻辑值
self	表示当前对话框
screen	表示用户屏幕，提供屏幕尺寸，颜色深度等信息
navigator	表示浏览器对象，用于获得与浏览器相关的信息

(2) window 对象的方法

方法	说明
alert ()	弹出一个警告对话框
confirm ()	在确认对话框中显示指定的字符串
prompt ()	弹出一个提示对话框
open ()	打开新浏览器对话框，并且显示有URL或名字引用的文档，并设置创建对话框的属性
close ()	关闭被引用的对话框
focus ()	将被引用的对话框放在所有打开对话框的前面
blur ()	将被引用的对话框放在所有打开对话框的后面
scrollTo (x, y)	把对话框滚动到指定的坐标
scrollBy (x, y)	按照指定的位移量滚动对话框
setTimeout (timer)	在指定毫秒后，对传递的表达式求值
setInterval (interval)	指定周期性执行代码
moveTo (x, y)	将对话框移动到指定坐标处
moveBy (offsetx, offsety)	将对话框移动到指定的位移量处
resizeTo (x, y)	设置对话框大小
resizeBy (offsetx, offsety)	按照指定的位移量设置对话框的大小
print ()	相当于浏览器工具栏中“打印”按钮
status ()	状态条，位于对话框下部的信息条，用于任何时间内信息的提示（有些浏览器不支持）
defaultstatus ()	状态条，位于对话框下部的信息条，用于某个事件发生时的信息的提示（有些浏览器不支持）

1. window 对象的消息提示框

window 对象常用的消息提示框，主要有以下 3 个，也是该对象的常用三个方法：

```
<body>
  <script>
    var a=window.confirm(); // 用户做出选择，选择结果保存在 a 中
    window.alert("提示框: a="+a); // 网页上出现消息提示框
    var b=window.prompt("请输入你的 id:"); // 让用户输入信息
    console.log(b);
  </script>
</body>
```

2. 打开和关闭窗口

window 对象还用于控制窗口的状态和开/关。打开窗口主要是 open（）函数，该函数有三个参数，第 1 个是窗口的文件地址，第 2 个是新窗口的名称，第 3 个是窗口的状态。其中第 3 个参数还可设置如下属性：

- toolbar: 是否有工具栏，值为 0 和 1；
- location: 是否有地址栏，值为 0 和 1；
- status: 是否有状态栏，值为 0 和 1；
- menubar: 是否有菜单栏，值为 0 和 1；
- scrollbars: 是否有滚动栏，值为 0 和 1；
- resizable: 是否能改变大小，值为 0 和 1；
- width 和 height: 窗口的高度和宽度，用像素表示。
- left 和 top: 窗口左上角相对于桌面左上角的 x 和 y。

下面用一个例子，在当前网页中打开一个已经存在的网页 `page2.html`，并设置其窗口大小和位置。

```
<body>
  <script>
    window.status="出现新窗口";
    // 注意有些浏览器可能会拦截新打开的窗口，需设置允许打开
    newWindow=window.open("page2.html","new1","width=300,height=300
, top=500, left=500, menubar=1, toolbar=1");
    //newWindow.close(); // 关闭窗口
  </script>
</body>
```

例题：实现用户注册页面，其中包含用户名，密码，确认密码文本框，还包含提交、重置、关闭按钮。当用户点击关闭按钮，将关闭当前浏览器。

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title> </title>
</head>
<body>
  <center>
    <form id="form4" name="form4" method="post" action="">
      <label> </label>
      <table width="353" height="140" border="1px solid">
        <tr>
          <td width="104" align="right">用户名: </td>
          <td width="233" align="left"><label
for="textfield"></label>
            <input type="text" name="textfield"
id="textfield" />
          </td>
        </tr>
        <tr>
          <td align="right">密码: </td>
          <td align="left"><label for="textfield2"></label>
            <input type="password" name="textfield2"
id="textfield2" />
          </td>
        </tr>
        <tr>
          <td align="right">确认密码: </td>
          <td align="left"><label for="textfield3"></label>
            <input type="password" name="textfield3"
id="textfield3" />
          </td>
        </tr>
        <tr>
          <td colspan="2" align="center"><label>
            <input type="submit" name="Submit" value="提
交" onclick="mysubmit()" />
          </label>
          <label>
            <input type="reset" name="Submit2" value="重
置" />
          </label>
          <label>

```

```

        <input type="button" name="Submit3" value="
关闭" onclick="window.close()" />
    </label>
</td>
</tr>
</table>
<label><br />
</label>
</form>
</center>
</body>
</html>

```

程序运行结果：

用户名：	
密码：	
确认密码：	
提交 重置 关闭	

4.6.2 String 对象

见 PPT。

4.6.3 Date 对象

在 Web 开发过程中，可以使用 JavaScript 的 Date 对象来对日期和时间进行操作。例如，如果想在网页中显示计时的时钟，就可以使用 Date 对象来获取当前系统的时间并按照指定的格式进行显示。

在 JavaScript 中提供 4 种构造函数方法，用于创建一个日期。

- `var time = new Date();`
- `var time = new Date(milliseconds);`
- `var time = new Date(datestring);`
- `var time = new Date(year, month, date[, hour, minute, second, millisecond]);`

参数说明如下：

- 不提供参数：若调用 Date() 函数时不提供参数，则创建一个包含当前时间和日期的 Date 对象；
- milliseconds（毫秒）：若提供一个数值作为参数，则会将这个参数视为一个以毫秒为单位的时间值，并返回自 1970-01-01 00:00:00 起，经过指定毫秒数的时间，例如 `new Date(5000)` 会返回一个 1970-01-01 00:00:00 经过 5000 毫秒之后的时间；

● **datestring** (日期字符串): 若提供一个字符串形式的日期作为参数, 则会将其转换为具体的时间, 日期的字符串形式有两种, 如下所示:

(1) **YYYY/MM/dd HH:mm:ss** (推荐): 若省略时间部分, 则返回的 **Date** 对象的时间为 **00:00:00**;

(2) **YYYY-MM-dd HH:mm:ss**: 若省略时间部分, 则返回的 **Date** 对象的时间为 **08:00:00** (加上本地时区), 若不省略, 在 **IE** 浏览器中会转换失败。

● 将具体的年月日、时分秒转换为 **Date** 对象, 其中:

(1) **year**: 表示年, 为了避免错误的产生, 推荐使用四位的数字来表示年份;

(2) **month**: 表示月, 0 代表 1 月, 1 代表 2 月, 以此类推;

(3) **date**: 表示月份中的某一天, 1 代表 1 号, 2 代表 2 号, 以此类推;

(4) **hour**: 表示时, 以 24 小时制表示, 取值范围为 0 ~ 23;

(5) **minute**: 表示分, 取值范围为 0 ~ 59;

(6) **second**: 表示秒, 取值范围为 0 ~ 59;

(7) **millisecond**: 表示毫秒, 取值范围为 0 ~ 999。

Date 对象常用方法:

方法	描述
<code>getDate()</code>	从 Date 对象返回一个月中的某一天 (1 ~ 31)
<code>getDay()</code>	从 Date 对象返回一周中的某一天 (0 ~ 6)
<code>getMonth()</code>	从 Date 对象返回月份 (0 ~ 11)
<code>getFullYear()</code>	从 Date 对象返回四位数字的年份
<code>getYear()</code>	已废弃, 请使用 <code>getFullYear()</code> 方法代替
<code>getHours()</code>	返回 Date 对象的小时 (0 ~ 23)
<code>getMinutes()</code>	返回 Date 对象的分钟 (0 ~ 59)
<code>getSeconds()</code>	返回 Date 对象的秒数 (0 ~ 59)
<code>getMilliseconds()</code>	返回 Date 对象的毫秒 (0 ~ 999)
<code>getTime()</code>	返回 1970 年 1 月 1 日至今的毫秒数

例题: 在网页中实时显示系统时间。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>实时显示系统时间</title>
</head>
```

```

<body>
  <div id="clock"></div>
  <script language="javascript">
    function realSysTime(clock) {
      var now = new Date();           //创建 Date 对象
      var year = now.getFullYear();   //获取年份
      var month = now.getMonth();     //获取月份
      var date = now.getDate();        //获取日期
      var day = now.getDay();          //获取星期
      var hour = now.getHours();       //获取小时
      var minu = now.getMinutes();     //获取分钟
      var sec = now.getSeconds();      //获取秒钟
      month = month + 1;
      var arr_week = new Array("星期日", "星期一", "星期二", "星期三", "星期四", "星期五", "星期六");
      var week = arr_week[day];        //获取中文的星期
      var time = year + "年" + month + "月" + date + "日 " + week
      + " " + hour + ":" + minu + ":" + sec; //组合系统时间
      clock.innerHTML = "当前时间: " + time; //显示系统时间
    }
    window.onload = function () {
      window.setInterval("realSysTime(clock)", 1000); //实时获取并
      显示系统时间
    }
  </script>
</body>
</html>

```

运行结果:



这里也用到了一个定时器 `setInterval()` 函数。

4.6.4 定时器

- (1) `setTimeout(function, 毫秒值)`: 在一定的时间间隔后执行一个 `function`, 只执行一次。
- (2) `setInterval(function, 毫秒值)`: 在一定的时间间隔后执行一个 `function`, 循环执行。

案例: 每隔一秒钟切换灯泡的状态。

```


<script>
  function on(){

```

```

        document.getElementById("myImage").src="../images/开灯.png";
    }
    function off(){
        document.getElementById("myImage").src="../images/关灯.png";
    }
    // 引入定时器
    var x=0;
    setInterval(function () {
        if (x++%2==0){
            on();
        }else {
            off();
        }
    },1000)
</script>

```

4.6.5 history 对象

history 对象包含用户的浏览历史信息，使用这个对象是因为它可以代替前进和后退按钮并访问历史记录，该对象从属于 window 对象。history 对象常用的函数如下：

history.back(): 返回上一页，相当于浏览器上的后退按钮。

history.forward(): 返回下一页，相当于单击浏览器上的前进按钮。

history.go(n): n 为整数，正数向前进 n 个页面，负数则后退 n 个页面。

举例，示例代码：

```

<body>
    <a onclick="history.forward()">前进</a>
    <a onclick="history.back()">后退</a>
</body>

```

4.6.6 location 对象

location 对象可以访问浏览器的地址栏，它也从属于 window 对象，其最常用的功能是跳转到另一个页面，调转的方法是修改 location 对象的 href 属性。

下面的例子单机按钮和超级链接，都能实现跳到图片的效果。

```

<body>
    <a href="./开灯.png">到图片</a>
    <input type="button" onclick="changloc()" value="按钮">
    <script>
        function changloc(){
            window.location.href="./开灯.png";
        }
    </script>

```


</body>

4.6.7 document 对象

document 对象从属于 window 对象，见 4.7DOM 技术。

4.7 DOM 技术

DOM 是 Document Object Model 的简称，即文档对象模型，表示文档（如 html 文档）和访问、操作构成文档的各种元素（如 html 标签和文本串）的应用程序接口 API。它提供了文档中独立元素的结构化、面向对象的表示方法，并允许通过对象的属性和方法访问这些对象。另外，文档对象模型还提供了添加、删除文档对象的方法，这样能够创建动态文档内容。DOM 也提供了处理事件的接口，它允许捕获和响应用户以及浏览器的动作。

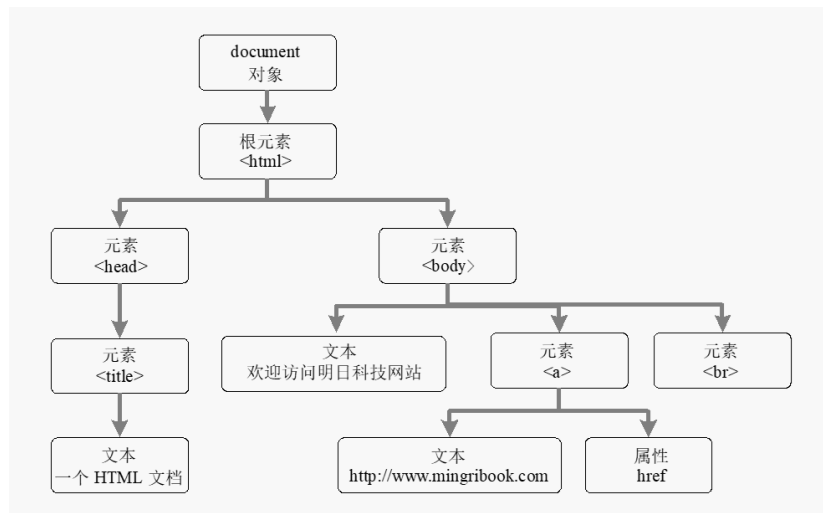
4.7.1 DOM 的分层结构

在 DOM 中，文档的层次结构以树形表示，树是倒立的，树根在上，枝叶在下。树的结点表示文档的内容。DOM 的根节点是整个 Document 对象，该对象的 documentElement 属性应用表示文档元素的 Element 对象。对于 HTML 文档，表示文档根元素的 Element 对象是<html>标签，<head>和<body>元素是树的枝干。

例题：一个简单的 HTML 文档对应的 DOM 分层结构。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>安徽大学</title>
</head>
<body>
  <h1>互联网学院</h1>
  <a href="https://www.ahu.edu.cn/">安徽大学主页</a>
</body>
</html>
```

上面的 HTML 文档的运行结果如下图所示，对应得 Document 对象的层次结构如下：



4.7.2 DOM 遍历文档

在 DOM 中，HTML 文档中的各个节点被视为各种类型的 Node 对象，并且将 HTML 文档表示为 Node 对象的树。对于任何一个树形结构来说，最常用的场合就是遍历树。在 DOM 中，可以通过 Node 对象的 parentNode, firstNode, nextNode, lastNode, previousSiling 等属性来遍历文档树。Node 对象的常用属性如下表所示。

nodeName	String	节点的名字；根据节点的类型而定义
nodeValue	String	节点的值；根据节点的类型而定义
nodeType	Number	节点的类型常量值之一
ownerDocument	Document	指向这个节点所属的文档
firstChild	Node	指向在childNodes列表中的第一个节点
lastChild	Node	指向在childNodes列表中的最后一个节点
childNodes	NodeList	所有子节点的列表
parentNode	Node	返回一个给定节点的父节点。
previousSibling	Node	指向前一个兄弟节点；如果这个节点就是第一个兄弟节点，那么该值为null
nextSibling	Node	指向后一个兄弟节点；如果这个节点就是最后一个兄弟节点，那么该值为null
hasChildNodes()	Boolean	当childNodes包含一个或多个节点时，返回真
attributes	NamedNodeMap	包含了代表一个元素的特性的Attr对象；仅用于Element节点
appendChild(node)	Node	将node添加到childNodes的末尾
removeChild(node)	Node	从childNodes中删除node
replaceChild(newnode, oldnode)	Node	将childNodes中的oldnode替换成newnode
insertBefore(newnode, refnode)	Node	在childNodes中的refnode之前插入newnode

遍历 4.7.1 节中 HTML 文档，并获取该文档中的全部标签及标签总数。

代码如下：

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

    <title>安徽大学</title>
</head>
<body onload="show()">
    <h1>互联网学院</h1>
    <a href="https://www.ahu.edu.cn/">安徽大学主页</a>
    <script language="javascript">
        var elementList = "";           //全局变量，保存 Element 标记名，使
用完毕要清空
        function getElement(node) {     //参数 node 是一个 Node 对象
            var total = 0;
            if (node.nodeType == 1) {    //检查 node 是否为 Element 对象
                total++;                 //如果是，计数器加 1
                elementList = elementList + node.nodeName + ",";           //
保存标记名
            }
            var childrens = node.childNodes;           //获
取 node 的全部子节点
            for (var m = node.firstChild; m != null; m = m.nextSibling)
            {
                total += getElement(m);           //对
每个子节点进行递归操作
            }
            return total;
        }
        function show() {
            var number =
getElement(document);           //获取标记总数
            elementList = elementList.substring(0, elementList.length -
1); //去除字符串中最后一个逗号
            alert("该文档中包含: " + elementList + "等" + number + "个标
记!");
            elementList =
            "";           //清空全局变量
        }
    </script>
</body>
</html>

```

3.7.3 获取文档中的指定元素（非常重要）

- getElementById(); 通过元素的 ID 属性获取元素节点对象；
- getElementsByName(); 通过元素 name 属性获取多个元素节点对象；
- getElementsByTagName(); 通过元素的标签名称获取多个元素节点对象；
- getElementsByClassName(); 通过元素的 class 属性获取多个元素节点对象；

注：这些都是 document 对象的方法，对于多个节点元素，返回的是一个数组，只有 id 属性是唯一的，因此无需使用数组接收。

举例：通过四个按钮，调用上述四种方法，实现元素 h1 标签的字体颜色改变

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>DoM 技术</title>
  <script>
    function changeBlue(){
      // 获取文档元素对象，通过 id 属性
      var elements=document.getElementById("id_h1");
      elements.style.color='red';
    }
    function changeGreen(){
      // 获取文档元素对象，通过 name 属性
      var elements=document.getElementsByName("name_h1");
      elements[0].style.color="green";
    }
    function changeRed(){
      // 获取文档元素对象，通过 tag name 属性
      var elements=document.getElementsByTagName("h1");
      elements[0].style.color="red";
    }
    function changeYellow(){
      // 获取文档元素对象，通过 class name 属性
      var elements=document.getElementsByClassName("cls_h1");
      //elements[0].style.color="yellow";
      // 除了 id 属性之外，其它都返回的是一个数组，因此标准写法应该借助
for 循环
      for (var i=0; i < elements.length; i++) {
        elements[i].style.color="yellow";
      }
    }
  </script>
</head>
<body>
  <h1 id="id_h1" name="name_h1" class="cls_h1" style="color: black;">
安徽大学互联网学院</h1>
  <button onclick="changeBlue()">蓝色-id</button>
  <button onclick="changeGreen()">绿色-name</button>
```

```
<button onclick="changeRed()">红色-TagName</button>
<button onclick="changeYellow()">黄色-ClassName</button>
</body>
</html>
```

3.7.4 操作文档（应用）

在 DOM 中不仅可以通过节点的属性查询节点，还可以对节点进行创建、插入、删除和替换等操作。这些操作都可以通过节点 Node 对象提供的方法来完成。Node 对象的常用方法如下表所示。

方 法	描 述
insertBefore(newChild,refChild)	在现有节点refChild之前插入节点newChild
replaceChild(newChild,oldChild)	将子节点列表中的子节点oldChild换成newChild，并返回oldChild节点
removeChild(oldChild)	将子节点列表中的子节点oldChild删除，并返回oldChild节点
appendChild(newChild)	将节点newChild添加到该节点的子节点列表末尾。如果newChild已经在树中，则先将其删除
hasChildNodes()	返回一个布尔值，表示节点是否有子节点
cloneNode(deep)	返回这个节点的副本（包括属性）。如果deep的值为true，则复制所有包含的节点；否则只复制这个节点

例题：通过两个按钮，实现插入一个标签和删除一个标签操作。

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM 增加和删除标签示例</title>
</head>
<body>
  <h1 id="myH1" class="name_h1">安徽大学互联网学院</h1>
  <button onclick="addH1()">增加</button>
  <button onclick="removeH1()">删除</button>
  <script>
    // 增加标题函数
    function addH1() {
      var h = document.createElement('h1');
      h.innerText = "安徽大学互联网学院";
      h.className="name_h1";
      document.body.appendChild(h);
    }
    // 删除标题函数
    function removeH1() {
      var h=document.getElementsByClassName("name_h1");
      console.log(h.length);
    }
  </script>
</body>
</html>
```

```

        if (h.length>0) {
            for (var i = 0; i < h.length; i++) {
                h[i].parentNode.removeChild(h[i]);
            }
        }
    }
</script>
</body>
</html>

```

例题：用 DOM 操作文档，实现添加评论和删除评论的功能。



人生若真如一场大梦，这个梦倒也很有趣的。在这个大梦里，一定还有长长短短，深深浅浅，肥肥瘦瘦、甜甜苦苦，无数的小梦。有些已经随着日影飞去；有些还远着哩.....

评论人	评论内容
评论人：	
评论内容：	

[发表](#)
[重置](#)
[删除第一条评论](#)
[删除最后一条评论](#)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>发表评论</title>
    <script language="javascript">
        function addElement() {
            //创建 TextNode 节点
            var person = document.createTextNode(form1.person.value);
            var content = document.createTextNode(form1.content.value);
            //创建 td 类型的 Element 节点
            var td_person = document.createElement("td");
            var td_content = document.createElement("td");
            var tr = document.createElement("tr"); //创建一个 tr 类型的
            Element 节点
            var tbody = document.createElement("tbody"); //创建一个 tbody
            类型的 Element 节点

```

```

        //将 TextNode 节点加入到 td 类型的节点中
        td_person.appendChild(person);
        td_content.appendChild(content);
        //将 td 类型的节点添加到 tr 节点中
        tr.appendChild(td_person);
        tr.appendChild(td_content);
        tbody.appendChild(tr); //将 tr 节点加入 tbody 中
        var tComment = document.getElementById("comment"); //获取
table 对象
        tComment.appendChild(tbody); //将节点 tbody 加入节点尾部
        form1.person.value = ""; //清空评论人文本框
        form1.content.value = ""; //清空评论内容文本框
    }
    //删除第一条评论
    function deleteFirstE() {
        var tComment = document.getElementById("comment"); //获取
table 对象
        if (tComment.rows.length > 1) {
            tComment.deleteRow(1); //删除表格的第二行，即第一条评论，
        }
    }
    //删除最后一条评论
    function deleteLastE() {
        var tComment = document.getElementById("comment"); //获取
table 对象
        if (tComment.rows.length > 1) {
            tComment.deleteRow(tComment.rows.length - 1); //删除表格的最后一行即最后一条评论
        }
    }
</script>
</head>

<body>
    <table width="600" height="70" border="0" align="center"
cellpadding="0" cellspacing="1" bordercolorlight="#FF9933"
bordercolordark="#FFFFFF" bgcolor="#666666">
        <thead>
            <tr>
                <td width="14%" align="center" bgcolor="#FFFFFF"></td>
                <td width="86%" align="left" bgcolor="#FFFFFF">

```



```
        &nbsp;
        <input name="Button" type="button" class="btn_grey"
value="删除第一条评论" onClick="deleteFirstE()">
        &nbsp;
        <input name="Button" type="button" class="btn_grey"
value="删除最后一条评论" onClick="deleteLastE()">
    </td>
</tr>
</table>
</form>
</body>
</html>
```